Modultestplan

Faktura — Desktop-Fakturierungsanwendung

Lucas Strubel

Version 3.1 · 10.10.2026

Inhaltsverzeichnis

Dokumentenhistorie 2
1. Einleitung 2
2. Testfälle 3
3. Ergänzte Testfälle ab Version 2.0 10
4. Testumfang-Übersicht 15
5. Abkürzungen 17

Ergänzung 3.1.0-rc.1 (10.10.2026) 17
RC-Regressionen und Freigabeläufe 17
Portfolio-Prüfungen (10.10.2026) 19

Autor	Rolle	Stand
Lucas Strubel	Verfasser für das SE1-Projektteam (Gesamtsystem)	15.06.2026 (v2.0), 24.09.2026 (v3.0)

Freigabevermerk: Dieses Dokument ist nach Prüfung und Freigabe durch den Modulverantwortlichen verbindliche Grundlage für den Modultest des Gesamtsystems *Desktop-Fakturierungsanwendung* (Komponenten A–D, gemeinsame Infrastruktur sowie Performance-Nachweise).

Dokumentenhistorie

Version	Datum	Autor	Grund der Änderung
1.0	15.06.2026	Lucas Strubel	Initiale Erstellung (ausgegliedert aus dem Pflichtenheft, Teil A)
2.0	15.06.2026	Lucas Strubel	Konsolidierung aller Komponenten-Testfälle (A–D, gemeinsame Infrastruktur, Performance) in einen projektweiten Modultestplan
3.0	24.09.2026	Lucas Strubel	Kapitel 3: ergänzte Testfälle der Versionen 2.0 und 3.0 (164 Testmethoden, 201 Ausführungen); Testumfang-Übersicht auf Stand 3.0
3.1	10.10.2026	Lucas Strubel	Review-Remediation 3.1.0-rc.1; historische Nachweise und offene Freigabeschranken getrennt

1. Einleitung

1.1 Zweck des Dokuments

Dieser Modultestplan spezifiziert die Modul-/Komponententests des **Gesamtsystems** *Desktop-Fakturierungsanwendung*. Er fasst die Testfälle aller vier Komponenten (A – Prozess/Dokumentenzyklus, B – Produktverwaltung, C – Kundenverwaltung, D – Programmoberfläche), der gemeinsam genutzten Infrastruktur (Paket `gemeinsam`) sowie die übergreifenden Performance-/Lastnachweise in einem Dokument zusammen. Die Testfälle leiten sich aus den funktionalen Anforderungen, Daten/Schnittstellen und testbaren Abnahmekriterien der jeweiligen Pflichtenhefte ab und überführen diese in deterministische, mit JUnit 5 umgesetzte Testfälle. Quelle der Wahrheit sind die Testklassen unter `src/test/java/de/lucasstrubel/faktura/…`.

> **Stand:** Kapitel 2 beschreibt den Testumfang der Abnahme von **Version 1.0** (71 Testfälle, 06/2026) und bleibt als Beleg unverändert. Die mit den Versionen 2.0 und 3.0 ergänzten Testfälle sind in Kapitel 3 aufgeführt; die Suite umfasst heute 164 Testmethoden mit 201 Ausführungen.

1.2 Rahmenbedingungen

Die folgenden Testfälle sind deterministisch (feste Ein-/Ausgaben) und mit JUnit 5 umgesetzt. Geldbeträge werden als `java.math.BigDecimal` mit **Scale 2** erwartet (`assertEquals(new BigDecimal("119.00"), …)` bzw. `compareTo`). Die jeweils benachbarten Komponenten werden im Modultest durch **Stubs/Mocks** ersetzt:

- **Komponente A** ersetzt die Schnittstellen der Komponenten B (Produkte) und C (Kunden) sowie den PDF-Export durch Stubs.
- **Komponente B** ersetzt die Schnittstelle `ProduktReferenzPruefung` (Komponente A) durch einen Stub.
- **Komponente C** ersetzt die Schnittstelle `KundenReferenzPruefung` (Komponente A) durch einen Stub.
- **Komponente D** ersetzt die Service-Schnittstellen der Komponenten A–C durch Stubs/Mocks.

Die Abkürzungen für die abgedeckten Pflichtenheft-Anforderungen (Spalte *Abgedeckte PH-Anf.*) sind **komponentenlokal** zu lesen: `F-01` in Abschnitt 2.2 bezeichnet eine Anforderung des Pflichtenhefts B, nicht des Pflichtenhefts A.

2. Testfälle

2.1 Komponente A — Prozess / Dokumentenzyklus

2.1.1 Dokumentenzyklus (`DokumentzyklusTest`)

TC	Abgedeckte PH-Anf.	Vorbedingung	Eingabe	Erwartetes Ergebnis
TC-01	F-23, F-03	Position mit Netto 100.00 €, Steuersatz 0.19	`berechne()`	Steuer = 19.00, Brutto = 119.00 (Scale 2)
TC-02	F-23	Einzelpreis 50.00 €, Menge 3	Positionssumme berechnen	positionssummeNetto = 150.00
TC-03	F-03, F-13	Beleg mit 2 Positionen (150.00 € @ 0.19; 50.00 € @ 0.07)	Summen berechnen	summeNetto = 200.00, summeSteuer = 32.00, summeBrutto = 232.00
TC-04	F-12, GR-01	Letzte Rechnungsnummer `R-2026-000123`	`naechsteNummer(RECHNUNG, 2026)`	liefert `R-2026-000124` (lückenlos)
TC-05	F-12 (Format)	Zähler = 7, Jahr 2026	`naechsteNummer(RECHNUNG, 2026)`	liefert `R-2026-000007` (führende Nullen, `String`)
TC-06	F-14, GR-06	Rechnungsdatum 2026-06-09, kein Zahlungsziel	Rechnung erstellen	zahlungsziel = 2026-06-23 (+14 Tage)
TC-07	F-14	Rechnungsdatum 2026-06-09, Zahlungsziel 2026-07-31	Rechnung erstellen	zahlungsziel = 2026-07-31 (übernommen)
TC-08	F-24, NF-INT-01	Rechnung im Status `VERSENDET`	`setzePositionen(...)` / Änderung	wirft `IllegalStateException`
TC-09	F-19, F-20	Rechnung im Status `OFFEN`	`storniere()`	Status = `STORNIERT`; nicht in `offeneRechnungen()`; `storniertAm` und `storniertVon` gesetzt

TC	Abgedeckte PH-Anf.	Vorbedingung	Eingabe	Erwartetes Ergebnis
TC-10	F-22, GR-05	Angebot `AN-2026-000001` mit Kunde + 2 Positionen	`erzeugeFolgebeleg(angebot)` (AB erzeugen)	AB übernimmt Kunde/Positionen/Mengen; `vorgaengerNr` = `"AN-2026-000001"`
TC-11	F-23, F-24	Rechnung mit Produkt @ 50.00 €; danach Produktpreis → 80.00 €	erste Rechnung erneut lesen	einzelpreisNetto bleibt 50.00 (Snapshot unverändert)
TC-12	F-18, NF-USE-02	Rechnung ohne Kunde **oder** ohne Position	`erstelleRechnung(...)`	Speichern abgelehnt; Validierungsfehler benennt fehlendes Pflichtfeld (`Kunde` bzw. `Position`)
TC-13	F-11, F-12, F-13	Kunde + 1 Position vorhanden	vollständige Rechnung erstellen	Rechnung gespeichert, Nummer `R-2026-…` vergeben, alle § 14 UStG-Pflichtangaben gesetzt

2.1.2 Belegpersistenz (`JsonDokumentRepositoryTest`)

TC	Abgedeckte PH-Anf.	Vorbedingung	Eingabe	Erwartetes Ergebnis
TC-14	IF-01 (GoBD)	Repository mit gespeicherter Rechnung `R-2026-000001` (OFFEN) und Angebot `AN-2026-000001` (ENTWURF)	Repository neu instanziieren (Neustart)	beide Belege geladen (`alle().size() == 2`); polymorphe Typen `Rechnung`/`Angebot` bleiben erhalten; Positionen ≠ `null`

2.1.3 PDF-Export (`PdfBoxPdfExporterTest`)

TC	Abgedeckte PH-Anf.	Vorbedingung	Eingabe	Erwartetes Ergebnis
TC-15	F-15, IF-01	Vollständige Rechnung `R-2026-000001`	`exportiere(rechnung, ziel)`	PDF-Datei existiert am Zielpfad
TC-16	F-15 (Robustheit)	Rechnung `R-2026-000099` mit Altdaten-Position (`produktReferenz`/`einzelpreisNetto`/`positionssummeNetto` = `null`)	`exportiere(rechnung, ziel)`	kein Fehler (`assertDoesNotThrow`); PDF-Datei existiert
TC-17	F-23 (Robustheit)	Rechnung `R-2026-000098` mit `null`-Positionssummen	Summen berechnen	summeNetto = summeSteuer = summeBrutto = 0.00 (Scale 2)

2.1.4 CSV-Datenexport der Belege (`DokumentCsvExportTest`)

TC	Abgedeckte PH-Anf.	Vorbedingung	Eingabe	Erwartetes Ergebnis
TC-18	Q-08, IF-04	Repository mit 2 Rechnungen (je 1 Position); eine storniert (Datum 2026-06-10, Benutzer „Anwender“)	`exportiereCsv(ziel)`	CSV mit Kopfzeile `belegnummer;belegtyp;datum;status…` + je Position eine Zeile (3 Zeilen); enthält `R-2026-000001` sowie `STORNIERT`/`Anwender`

2.2 Komponente B — Produktverwaltung

PH-Anf.-Nummern beziehen sich auf Teil B des Pflichtenhefts. Die Schnittstelle `ProduktReferenzPruefung` (Komponente A) wird durch einen Stub ersetzt.

TC	Abgedeckte PH-Anf.	Vorbedingung	Eingabe	Erwartetes Ergebnis
TC-01	F-01, F-02	Höchste Produktnummer `P-000041`	Produkt („Beratungsstunde“, 80.00, 0.19) speichern	Produkt persistiert; Produktnummer = `P-000042`
TC-02	F-02 (Format)	Zähler = 7	`naechsteNummer()`	liefert `P-000007` (führende Nullen, `String`)
TC-03	F-03	gültiges Produkt	Einzelpreis `-1.00`	Speichern abgelehnt (Validierungsfehler „Einzelpreis“)
TC-04	F-03	gültiges Produkt	Steuersatz `0.15`	Speichern abgelehnt (unzulässiger Steuersatz)
TC-05	F-04, NF-USE-01	Produkt ohne Bezeichnung	`speichere()`	Speichern abgelehnt; Validierungsfehler benennt „Bezeichnung“
TC-06	F-05	Produkt `P-000042` mit Preis 80.00	Preis auf 95.00 ändern, speichern	gespeichertes Produkt hat einzelpreisNetto = 95.00
TC-07	F-07	Produkt `P-000042`	Änderungsversuch der Produktnummer auf `P-999999`	wirft `IllegalArgumentException` / Änderung abgelehnt
TC-08	F-08	Produkt unverknüpft (Stub: `istProduktReferenziert` → `false`)	`loescheProdukt("P-000011")` mit Bestätigung	Produkt entfernt; nicht mehr in `alleSortiertNachBezeichnung()`
TC-09	F-09, F-10	Stub: `istProduktReferenziert("P-000010")` → `true`	`loescheProdukt("P-000010")`	Löschen abgelehnt; Produkt weiterhin vorhanden; Hinweis erzeugt
TC-10	F-11	Produkte „Zaun“, „Anker“, „Mast“	`alleSortiertNachBezeichnung()`	Reihenfolge: „Anker“, „Mast“, „Zaun“

TC	Abgedeckte PH-Anf.	Vorbedingung	Eingabe	Erwartetes Ergebnis
TC-11	F-12	Produkt „Beratungsstunde“	`suche("BERATUNG")`	Trefferliste enthält „Beratungsstunde“ (case-insensitive, Teilstring)
TC-12	F-12	Produkt `P-000042`	`suche("P-000042")`	Trefferliste enthält genau dieses Produkt
TC-13	F-14	Kein Produkt `P-999999` vorhanden	`findeProdukt("P-999999")`	liefert `null`
TC-14	F-15	3 Produkte im Bestand	`exportiereCsv(ziel)`	CSV-Datei mit Kopfzeile + 3 Datenzeilen, Semikolon-getrennt, UTF-8

2.3 Komponente C — Kundenverwaltung

PH-Anf.-Nummern beziehen sich auf Teil C des Pflichtenhefts. Die Schnittstelle `KundenReferenz Pruefung` (Komponente A) wird durch einen Stub ersetzt.

TC	Abgedeckte PH-Anf.	Vorbedingung	Eingabe	Erwartetes Ergebnis
TC-01	F-01, F-02	Höchste Kundennummer `K-000016`	Kunde („Muster GmbH“, „Hauptstr. 1“, „68163“, „Mannheim“) speichern	Kunde persistiert; Kundennummer = `K-000017`
TC-02	F-02 (Format)	Zähler = 7	`naechsteNummer()`	liefert `K-000007` (führende Nullen, `String`)
TC-03	F-03, NF-USE-01	Kunde ohne Ort	`speichere()`	Speichern abgelehnt; Validierungsfehler benennt „Ort“
TC-04	F-03	Kunde mit leerem Namen (`""`)	`speichere()`	Speichern abgelehnt; Validierungsfehler benennt „Name“
TC-05	F-04	Kunde mit E-Mail `"max.mustermann"`	`speichere()`	Speichern abgelehnt (ungültiges E-Mail-Format)
TC-06	F-04	Kunde mit E-Mail `"max@beispiel.de"`	`speichere()`	Kunde gespeichert (gültiges Format)
TC-07	F-05	Kunde `K-000017` mit Ort „Mannheim“	Ort auf „Heidelberg“ ändern, speichern	gespeicherter Kunde hat ort = „Heidelberg“
TC-08	F-07	Kunde `K-000017`	Änderungsversuch der Kundennummer auf `K-999999`	wirft `IllegalArgument Exception` / Änderung abgelehnt
TC-09	F-08	Stub: `anzahl Verknuepfter Dokumente` → `0`	`loescheKunde("K-000011")` mit Bestätigung	Kunde entfernt; nicht mehr in `alleSortiertNachName()`

TC	Abgedeckte PH-Anf.	Vorbedingung	Eingabe	Erwartetes Ergebnis
TC-10	F-09, F-10, GR-04	Stub: `anzahlVerknuepfterDokumente("K-000010")` → `3`	`loescheKunde("K-000010")`	Löschen abgelehnt; Kunde weiterhin vorhanden; Hinweis enthält Anzahl `3`
TC-11	F-11	Kunden „Zimmer“, „Albrecht“, „Maier“	`alleSortiertNachName()`	Reihenfolge: „Albrecht“, „Maier“, „Zimmer“
TC-12	F-12	Kunde „Muster GmbH“	`suche("MUSTER")`	Trefferliste enthält „Muster GmbH“ (case-insensitive, Teilstring)
TC-13	F-12, F-14	Kunde `K-000017` vorhanden; `K-999999` nicht	`suche("K-000017")`; `findeKunde("K-999999")`	Treffer enthält `K-000017`; `findeKunde` liefert `null`
TC-14	F-15	3 Kunden im Bestand	`exportiereCsv(ziel)`	CSV-Datei mit Kopfzeile + 3 Datenzeilen, Semikolon-getrennt, UTF-8

2.4 Komponente D — Programmoberfläche

PH-Anf.-Nummern beziehen sich auf Teil D des Pflichtenhefts. Getestet wird die GUI-freie Controller- und Modell-Schicht; die Service-Schnittstellen der Komponenten A–C werden durch Stubs/Mocks ersetzt.

TC	Abgedeckte PH-Anf.	Vorbedingung	Eingabe	Erwartetes Ergebnis
TC-01	F-09	Wizard neu gestartet	`aktuellerSchritt` lesen	`KUNDE_WAEHLEN` (erster Schritt)
TC-02	F-09	Schritt 1 mit gewähltem Kunden	`weiter()` 4-mal mit gültigen Eingaben	Schrittfolge: `POSITIONEN_ERFASSEN` → `DATEN_BESTAETIGEN` → `ZUSAMMENFASSUNG` → `SPEICHERN`
TC-03	F-10	Schritt 1, kein Kunde gewählt (`kundenNr = null`)	`weiter()`	Wechsel verhindert; `Meldung(FEHLER, "Kunde", …)` erzeugt
TC-04	F-10	Schritt 2, leere Positionsliste	`weiter()`	Wechsel verhindert; Meldung benennt „Position“
TC-05	F-10	Schritt 2, Position mit `menge = 0`	`weiter()`	Wechsel verhindert; Meldung benennt „Menge“
TC-06	F-11	Schritt 3 erreicht; Kunde `K-000017`, 1 Position erfasst	`zurueck()` bis Schritt 1	`kundenNr` und `positionen` unverändert erhalten

TC	Abgedeckte PH-Anf.	Vorbedingung	Eingabe	Erwartetes Ergebnis
TC-07	F-12	Schritt 4; Stub `DokumentService` liefert Summen 200.00/38.00/238.00	Zusammenfassung erzeugen	Zusammenfassung enthält Kunde, Positionen, Mengen, 200.00/38.00/238.00, Rechnungsdatum, Zahlungsziel
TC-08	F-13	Schritt 5; gültiges Modell	`speichern()`	genau **ein** Aufruf `erstelleRechnung(...)` am Mock; Erfolgsmeldung enthält gelieferte Rechnungsnummer
TC-09	F-13 (Fehlerfall)	Stub `erstelleRechnung` wirft Validierungsfehler „Rechnungsdatum“	`speichern()`	keine Erfolgsmeldung; `Meldung(FEHLER, "Rechnungsdatum", …)` dargestellt (F-05/F-16)
TC-10	F-14	Dokumentliste mit Rechnungen in `OFFEN`, `VERSENDET`, `STORNIERT`	verfügbare Aktionen je Rechnung ermitteln	*Stornieren* nur bei Status `OFFEN` aktiviert
TC-11	F-15	Rechnung `R-2026-000124`	`storniere()` ohne Bestätigung; danach mit Bestätigung	ohne Bestätigung: kein Service-Aufruf; mit Bestätigung: genau ein Aufruf `storniere("R-2026-000124")`
TC-12	F-08	Beleg im Status `VERSENDET`	Änderungsaktionen ermitteln	alle inhaltlichen Änderungsaktionen deaktiviert; PDF-Export aktiviert
TC-13	F-06	Belege mit Status `OFFEN` (2×) und `STORNIERT` (1×)	Statusfilter `OFFEN` anwenden	Liste enthält genau die 2 offenen Belege
TC-14	F-03	Stub `KundenService.suche("Muster")` liefert 1 Treffer	Suchbegriff „Muster“ eingeben	Controller delegiert an `KundenService.suche(...)`; Trefferliste enthält genau diesen Kunden
TC-15	F-03 (D-F-03)	Stub `KundenService` mit 1 Kunden „Muster GmbH“	`kundenListe("")`, `(" ")`, `(null)`, `("Muster")`, `("unbekannt")`	leerer/fehlender Suchbegriff: gesamter Bestand (1); „Muster“: 1 Treffer; „unbekannt“: 0 Treffer

2.5 Gemeinsame Infrastruktur (Paket `gemeinsam`)

Querschnittliche Dienste, die von allen Komponenten genutzt werden. Der `EreignisBus` (Observer-Muster) ist in der Architektur der Komponente A (Pflichtenheft A §7) verortet; `JsonPersistenz` ist die gemeinsame atomare JSON-Ablage hinter allen Repositories (IF-01).

2.5.1 Ereignisbenachrichtigung (`EreignisBusTest`)

TC	Abgedeckte Anf.	Vorbedingung	Eingabe	Erwartetes Ergebnis
INF-01	Observer (PH-A §7)	`EreignisBus` mit 2 Beobachtern für Bereich `KUNDEN`	`melde(KUNDEN)` 2×	beide Beobachter genau 2× benachrichtigt
INF-02	Observer	Beobachter für `KUNDEN` und `DOKUMENTE`	`melde(DOKUMENTE)`	nur `DOKUMENTE`-Beobachter (1×); `KUNDEN`-Beobachter 0×
INF-03	Observer (Robustheit)	`EreignisBus` ohne Beobachter	`melde(PRODUKTE)`	wirkungslos, keine Exception

2.5.2 Atomare JSON-Persistenz (`JsonPersistenzTest`)

TC	Abgedeckte Anf.	Vorbedingung	Eingabe	Erwartetes Ergebnis
INF-04	IF-01 (atomar)	leeres Verzeichnis	`schreibeAtomar(datei, ["a","b"])`	Zieldatei existiert; keine `.tmp`-Restdatei; Inhalt = `["a","b"]`
INF-05	IF-01	Datei mit Bestand `["alt"]`	`schreibeAtomar(datei, ["neu1","neu2"])`	Inhalt vollständig ersetzt = `["neu1","neu2"]`
INF-06	IF-01	Zielpfad mit fehlendem Unterordner	`schreibeAtomar(unterordner/datei, ["a"])`	fehlende Elternverzeichnisse angelegt; Datei existiert

2.6 Performance-/Lastnachweise (`PerformanceTest`)

Übergreifende Lastnachweise gemäß Lastenheft. Referenzgröße: 5.000 Kunden, 5.000 Produkte, 1.000 Belege (vorab geseedet; das Befüllen fließt nicht in die Messung ein). Die Spalte *Erwartetes Ergebnis* nennt die einzuhaltende Zeitschranke.

ID	Abgedeckte Anf.	Vorbedingung	Eingabe	Erwartetes Ergebnis (Zeitschranke)
Q-04	Q-04 (Start)	Seeding 5.000 Kunden / 5.000 Produkte / 1.000 Belege (je JSON)	drei Repositories laden	Laden in ≤ 5 s
Q-02	Q-02 (Suche)	Bestand wie Q-04	Kunden-/Produktsuche + Auflistung	abgeschlossen in ≤ 1 s
Q-03	Q-03 (PDF)	Rechnung mit 50 Positionen	`exportiere(rechnung, ziel)`	PDF-Erstellung in ≤ 2 s
Q-08	Q-08 (Export)	Bestand wie Q-04	Vollexport Kunden + Produkte + Belege als CSV	abgeschlossen in ≤ 30 s

3. Ergänzte Testfälle ab Version 2.0

Kapitel 2 beschreibt den Testumfang der Abnahme von Version 1.0 und bleibt als Beleg unverändert. Die folgenden Testfälle kamen mit den Versionen 2.0 und 3.0 hinzu; die Beschreibung entspricht dem `@DisplayName` der Testmethode. Anforderungs-IDs mit Präfix (z. B. `A-F-29`) verweisen auf die Weiterentwicklung im Pflichtenheft v2.1–v2.4.

Zwei Testfälle aus Kapitel 2 wurden mit Version 3.0 fachlich angepasst:

- **D / TC-10:** *Stornieren* ist jetzt für offene **und** versendete, unbezahlte Rechnungen aktiviert — eine versendete Rechnung wird über eine Stornorechnung ausgeglichen (A-F-29). Zusätzlich wird die Aktion *Als bezahlt markieren* geprüft (A-F-28).
- **D / TC-07:** Die Zusammenfassung zeigt die Beträge im deutschen Format (`238,00 €` statt `238.00`) und zusätzlich das Leistungsdatum (A-F-31).

3.1 A – Dokumentenzyklus: `DokumentzyklusTest`

ID	Testfall
DZ-01	Storno einer versendeten Rechnung erzeugt eine Stornorechnung mit negativen Mengen (A-F-29)
DZ-02	Eine offene Rechnung wird ohne Stornorechnung storniert; Stornorechnungen sind weder stornier- noch bezahlbar
DZ-03	Zahlungseingang einer versendeten Rechnung wird erfasst, der Inhalt bleibt gesperrt (A-F-28)
DZ-04	Zahlungsdatum ist Pflicht und darf nicht vor dem Rechnungsdatum liegen (A-F-28)
DZ-05	Rechnung aus Lieferschein übernimmt das Lieferdatum als Leistungsdatum (A-F-31)
DZ-06	Abweichendes Leistungsdatum wird übernommen; Zahlungsziel/Gültigkeit vor dem Belegdatum abgelehnt (A-F-31, A-F-32)
DZ-07	Belege speichern das Firmenprofil als Snapshot; spätere Änderungen wirken nicht zurück (A-F-27)
DZ-08	Ohne Firmenprofil bzw. ohne Steuerkennung kein Beleg — und keine verbrauchte Nummer (A-F-26, GR-01)
DZ-09	Liefert der Nummernkreis eine vergebene Nummer, wird nichts überschrieben (GR-01, GR-02)
DZ-10	Umsatzsteuer je Steuersatz — 7 % und 19 % getrennt ausgewiesen (A-F-25)

3.2 A – Dokumentenzyklus: `ERechnungExportTest`

ID	Testfall
ERE-01	EN-16931-XML ist wohlgeformt und trägt die Guideline-Kennung
ERE-02	Belegnummer, Parteien, Steuerkennungen und Bruttosumme stammen aus den Snapshots

ID	Testfall
ERE-03	Umsatzsteuer je Steuersatz gerundet — XML und Belegsumme stimmen überein (A-F-25)
ERE-04	Die Stornorechnung wird als korrigierte Rechnung (384) mit Verweis ausgegeben (A-F-29)
ERE-05	Eine stornierte Rechnung wird nicht mehr als E-Rechnung ausgestellt
ERE-06	Ein Komma in der Straßenangabe verschiebt die PLZ nicht
ERE-07	Der Aussteller-Snapshot hat Vorrang vor dem aktuellen Firmenprofil (A-F-27)
ERE-08	Ohne Steuernummer und USt-IdNr. wird keine E-Rechnung erzeugt (BR-CO-26)

3.3 A – Dokumentenzyklus: `JdbcDokumentRepositoryTest`

ID	Testfall
JDB-01	Rechnung mit Positionen wird verlustfrei gespeichert und geladen
JDB-02	versendete Rechnung bleibt nach dem Laden unveränderlich (GR-02)
JDB-03	stornierte Rechnung behält Stornodatum und -benutzer (BA-14)
JDB-04	alle vier Belegtypen werden über den Diskriminator korrekt geladen
JDB-05	unbekannte Belegnummer liefert null; Speichern aktualisiert per Upsert
JDB-06	Aussteller-Snapshot, Zahlungseingang und Stornoverweis überstehen die Rundreise (A-F-27 bis A-F-29)
JDB-07	Versendete Belege behalten ihre gespeicherten Summen, auch wenn die Rechenregel sich ändert (GR-02)

3.4 A – Dokumentenzyklus: `KennzahlenDienstTest`

ID	Testfall
KZ-01	Offene und versendete Rechnungen bilden den offenen Betrag
KZ-02	Überfällig ist nur, wessen Zahlungsziel vor dem Stichtag liegt (GR-06)
KZ-03	Der Jahresumsatz zählt nur nicht stornierte Rechnungen des laufenden Jahres
KZ-04	Die jüngsten Belege stehen zuerst und sind begrenzt
KZ-05	Ein leerer Bestand liefert Nullwerte statt einer Ausnahme
KZ-06	Bezahlte Rechnungen sind nicht mehr offen; Stornorechnungen zählen weder offen noch als Umsatz (A-F-28, A-F-29)

3.5 A – Dokumentenzyklus: `NummernkreisIntegritaetTest`

ID	Testfall
NK-01	Ein fehlgeschlagenes Speichern verbraucht keine Rechnungsnummer (GR-01)
NK-02	Ein ungültiger Beleg berührt den Nummernkreis nicht (F-18)
NK-03	Der Zähler übersteht einen Neustart und zählt lückenlos weiter (GR-01)

3.6 A – Dokumentenzyklus: `PdfBoxPdfExporterTest`

ID	Testfall
PDF-01	Namen außerhalb von WinAnsi (Ł, ő, Ş), Emoji und Tabulatoren brechen den Export nicht ab
PDF-02	Lange Bezeichnungen werden umbrochen statt abgeschnitten (§ 14 Abs. 4 Nr. 5 UStG)
PDF-03	Umsatzsteuer wird je Steuersatz mit Bemessungsgrundlage ausgewiesen (A-F-25)
PDF-04	Die Stornorechnung trägt ihren Titel, den Verweis und keinen Zahlungshinweis (A-F-29)
PDF-05	Ohne Aussteller-Snapshot und ohne Firmenprofil wird kein PDF erzeugt (A-F-26)
PDF-06	Die Anschrift wird an der letzten Trennstelle in Straße und PLZ/Ort geteilt

3.7 B – Produktverwaltung: `ProduktVerwaltungTest`

ID	Testfall
TC-15	Preise mit mehr als zwei Nachkommastellen werden abgelehnt statt still gerundet (B-F-03)
TC-16	Ändern eines nicht existierenden Produkts wird abgelehnt statt es anzulegen

3.8 C – Kundenverwaltung: `KundenVerwaltungTest`

ID	Testfall
TC-15	ungültige PLZ ‘123’ wird beim Anlegen abgelehnt und benannt (C-F-16)
TC-16	ungültige USt-IdNr. wird beim Anlegen abgelehnt und benannt (C-F-17)
TC-17	ungültiges Telefon wird beim Ändern abgelehnt und benannt (C-F-18)
TC-18	Eingaben werden getrimmt, leere optionale Felder null, USt-IdNr. normalisiert (C-F-19)
TC-19	Ändern eines nicht existierenden Kunden wird abgelehnt statt ihn anzulegen
TC-20	Liefert der Nummernkreis eine vergebene Nummer, wird kein Bestandskunde überschrieben

3.9 D – Programmoberfläche: `DokumentListenControllerTest`

ID	Testfall
DL-01	Ohne Filter erscheinen alle Belege
DL-02	Der Statusfilter grenzt auf den gewählten Status ein
DL-03	Die Suche findet Belegnummer, Kundenname und Kundennummer
DL-04	Die Suche achtet nicht auf Groß- und Kleinschreibung
DL-05	Status und Suche wirken zusammen
DL-06	Ein Suchbegriff ohne Treffer liefert eine leere Liste

ID	Testfall

3.10 D – Programmoberfläche: `OberflaechenControllerTest`

ID	Testfall
ZE-01	Zahlungseingang wird an die Fachlogik gegeben und als Erfolg gemeldet (A-F-28)
ZE-02	Datumsprüfung im Wizard — Zahlungsziel vor dem Rechnungsdatum wird abgelehnt (A-F-32)

3.11 D – Programmoberfläche: `TabellenFormatTest`

ID	Testfall
TF-01	Übliche deutsche und englische Schreibweisen werden eindeutig gelesen
TF-02	Mehrdeutige oder ungültige Eingaben werden abgelehnt statt still falsch gelesen

3.12 Gemeinsame Infrastruktur: `CsvTest`

ID	Testfall
CSV-01	Die Datei beginnt mit einer UTF-8-Byte-Order-Mark und behält Umlaute
CSV-02	Trennzeichen, Anführungszeichen und Zeilenumbrüche (auch einzelnes CR) werden maskiert
CSV-03	Formeln werden entschärft, Zahlen und Telefonnummern bleiben unverändert

3.13 Gemeinsame Infrastruktur: `DatensicherungTest`

ID	Testfall
DS-01	Die Sicherung enthält eine gültige Datenbank mit den aktuellen Daten
DS-02	Vorhandene JSON-Bestände werden mitgesichert, Logdateien nicht

3.14 Gemeinsame Infrastruktur: `ValidierungTest`

ID	Testfall
VAL-01	fehlendes Pflichtfeld wird abgelehnt und benannt (Q-09)
VAL-02	gültige fünfstellige PLZ wird akzeptiert (C-F-16)
VAL-03	ungültige PLZ wird abgelehnt und benannt (C-F-16)
VAL-04	gültige E-Mail-Adressen werden akzeptiert (C-F-04)
VAL-05	ungültige E-Mail-Adressen werden abgelehnt (C-F-04)
VAL-06	gültige USt-IdNr. (auch mit Leerzeichen) wird akzeptiert (C-F-17)

ID	Testfall
VAL-07	ungültige USt-IdNr. wird abgelehnt und benannt (C-F-17)
VAL-08	gültige Telefonnummern werden akzeptiert (C-F-18)
VAL-09	zu kurze oder unzulässige Telefonnummern werden abgelehnt (C-F-18)
VAL-10	leere optionale Felder (E-Mail, Telefon, USt-IdNr., PLZ-Format) werfen nicht
VAL-11	USt-IdNr. anderer EU-Staaten und Kleinschreibung werden akzeptiert (C-F-17)
VAL-12	IBAN mit falscher Prüfziffer wird abgelehnt, gültige auch klein und mit Leerzeichen akzeptiert (C-F-20)
VAL-13	BIC mit 8 oder 11 Zeichen wird akzeptiert, andere Längen abgelehnt (C-F-20)
VAL-14	Steuernummer mit 10 bis 13 Ziffern und Schrägstrichen (C-F-20)
VAL-15	Normalisierung — leere Werte werden null, USt-IdNr./IBAN/BIC einheitlich (C-F-19)

3.15 Firmenprofil: `FirmenprofilServiceTest`

ID	Testfall
FP-01	Ohne gespeichertes Profil gibt es kein Platzhalterprofil, und Belege werden abgelehnt (A-F-26)
FP-02	Ein gespeichertes Profil wird vollständig wieder geladen
FP-03	Erneutes Speichern ersetzt das Profil, statt ein zweites anzulegen
FP-04	Fehlende Pflichtfelder werden mit Feldbezug abgelehnt (Q-09)
FP-05	Ungültige Formate der optionalen Felder werden abgelehnt (C-F-20)
FP-06	plzOrt() setzt die einzeilige Anschrift für den Briefkopf zusammen
FP-07	Rechnungen verlangen Steuernummer oder USt-IdNr., andere Belege nicht (A-F-26)
FP-08	Eingaben werden einheitlich gespeichert — getrimmt, USt-IdNr./IBAN/BIC normalisiert (C-F-19)
FP-09	hatSteuerkennung() erkennt Steuernummer oder USt-IdNr.

3.16 Anwendung/Querschnitt: `FakturaApplicationTest`

ID	Testfall
KTX-01	Spring-Kontext startet und verdrahtet alle Fachkomponenten
KTX-02	SQLite-Datenbank liegt im konfigurierten Datenverzeichnis (IF-01)
KTX-03	JSON-Bestand wird einmalig übernommen, Nummernvergabe läuft weiter
KTX-04	Übernommene JSON-Dateien werden umbenannt und nie erneut eingelesen (GR-01)
KTX-05	Eine fehlerhafte JSON-Datei bricht die Übernahme vollständig ab; der nächste Start holt sie nach
KTX-06	Das Datenverzeichnis kommt aus dem Startargument, sonst aus dem Benutzerverzeichnis

4. Testumfang-Übersicht

4.1 Stand Version 3.0

Parametrisierte Tests (`@ParameterizedTest`) werden je Parametersatz einmal ausgeführt; daher liegt die Zahl der Ausführungen über der Zahl der Testmethoden.

Testklasse	Bereich	Testmethoden	davon seit 2.0	Ausführungen
`dokumente/DokumentCsvExportTest`	A – Dokumentenzyklus	1	0	1
`dokumente/DokumentzyklusTest`	A – Dokumentenzyklus	23	10	23
`dokumente/ERechnungExportTest`	A – Dokumentenzyklus	8	8	8
`dokumente/jdbc/JdbcDokumentRepositoryTest`	A – Dokumentenzyklus	7	7	7
`dokumente/json/JsonDokumentRepositoryTest`	A – Dokumentenzyklus	1	0	1
`dokumente/KennzahlenDienstTest`	A – Dokumentenzyklus	6	6	6
`dokumente/NummernkreisIntegritaetTest`	A – Dokumentenzyklus	3	3	3
`dokumente/pdf/BoxPdfExporterTest`	A – Dokumentenzyklus	9	6	9
`produkte/ProduktVerwaltungTest`	B – Produktverwaltung	16	2	16
`kunden/KundenVerwaltungTest`	C – Kundenverwaltung	20	6	20
`gui/dokument/DokumentListenControllerTest`	D – Programmoberfläche	6	6	6
`gui/OberflaechenControllerTest`	D – Programmoberfläche	17	2	17
`gui/TabellenFormatTest`	D – Programmoberfläche	2	2	16

Testklasse	Bereich	Testmethoden	davon seit 2.0	Ausführungen
`gemeinsam/CsvTest`	Gemeinsame Infrastruktur	3	3	3
`gemeinsam/DatensicherungTest`	Gemeinsame Infrastruktur	2	2	2
`gemeinsam/EreignisBusTest`	Gemeinsame Infrastruktur	3	0	3
`gemeinsam/JsonPersistenzTest`	Gemeinsame Infrastruktur	3	0	3
`gemeinsam/ValidierungTest`	Gemeinsame Infrastruktur	15	15	38
`firma/FirmenprofilServiceTest`	Firmenprofil	9	9	9
`FakturaApplicationTest`	Anwendung/Querschnitt	6	6	6
`PerformanceTest`	Anwendung/Querschnitt	4	0	4
Summe		**164**	**93**	**201**

4.2 Stand Version 1.0 (Abnahme)

Testklasse	Komponente	Anzahl
`dokumente/DokumentzyklusTest`	A – Prozess/Dokumentenzyklus	13
`dokumente/JsonDokumentRepositoryTest`	A – Belegpersistenz (IF-01/GoBD)	1
`dokumente/PdfBoxPdfExporterTest`	A – PDF-Export (F-15)	3
`dokumente/DokumentCsvExportTest`	A – CSV-Export Belege (Q-08)	1
`produkte/ProduktVerwaltungTest`	B – Produktverwaltung	14
`kunden/KundenVerwaltungTest`	C – Kundenverwaltung	14
`gui/OberflaechenControllerTest`	D – Programmoberfläche	15
`gemeinsam/EreignisBusTest`	Gemeinsame Infrastruktur (Observer)	3
`gemeinsam/JsonPersistenzTest`	Gemeinsame Infrastruktur (Persistenz)	3
`PerformanceTest`	Querschnitt (Q-02/Q-03/Q-04/Q-08)	4
Summe		**71**

Damit sind **71 Testfälle** über alle vier Komponenten, die gemeinsame Infrastruktur und die Performance-Nachweise spezifiziert. Sie decken die funktionalen Kernregeln, die zentralen Geschäftsregeln (GR-01…GR-06) sowie die Qualitäts-/Performanceanforderungen (Q-02, Q-03, Q-04, Q-08, Q-09) ab.

5. Abkürzungen

Abkürzung	Bedeutung
TC	Testfall (Test Case)
INF	Infrastruktur-Testfall (Paket `gemeinsam`)
F	Funktionale Anforderung (Pflichtenheft, komponentenlokal)
NF	Nicht-funktionale Anforderung (Pflichtenheft)
IF	Schnittstelle (Interface)
GR	Geschäftsregel (Lastenheft)
Q	Qualitätsanforderung (Lastenheft)
PH	Pflichtenheft
CSV	Comma-Separated Values (offenes Exportformat)
GoBD	Grundsätze zur ordnungsmäßigen Führung und Aufbewahrung von Büchern

Ergänzung 3.1.0-rc.1 (10.10.2026)

Maßgeblicher RC-Umfang: EN 16931 CII mit eingebetteter Schema-/Schematron-Prüfung; XRechnung ist nicht implementiert. Lückenlose Nummernvergabe ist Projektinvariante GR-01, keine pauschale Aussage über gesetzliche Anforderungen oder Compliance. Historische Beträge und unbekannte Einheiten werden erhalten; nicht reproduzierbares XML wird gesperrt. Details zu Snapshots, Migration, Einheiten, CSV und Rücksicherung stehen in Betrieb 3.1. Die v1.0-Abnahme bleibt ein historischer Nachweis. Aktuelle Ergebnisse, verbleibende Plattformprüfungen und offene Usability-Studien sind im Modultestbericht ausgewiesen. Frühere JSON-Zeitmessungen belegen weder SQLite-Performance noch die Startzeit des gepackten Programms.

RC-Regressionen und Freigabeläufe

ID	Nachweis
REM-01	SQLite-Preisrundreise mit 1,000, 12,500, Dezimal-/Tausenderformaten und Fremdfeldänderung
REM-02	Kalenderstrenge Kurzformen, Schaltjahre und feldbezogene Fehler
REM-03	Historische 0,08 EUR, PDF, Storno -0,08 EUR und Neustart
REM-04	V4-Migration mit verifizierter Sicherung und unveränderten Nummern
REM-05	Fehlende Beträge, Presets/Aliase und exakte Positionsnegation
REM-06	Gesamt-CSV, ZIP-Schreibfehler, Wiederherstellung und Integrität
REM-07	Alter JSON-Import mit fehlenden Geld-/Einheitenfeldern

ID	Nachweis
REM-08	Fehler der Upgradeprobe erhält V4; Wiederholung gelingt
REM-09	Geänderte Produkteinheit beeinflusst Folgebelege und Storno auch nach Neustart nicht
REM-10	Produkt ohne Einheit ergibt Stück/C62; Pauschale (LS) und Monat (MON) bestehen die EN-16931-Prüfung
REM-11	Eingefrorene V6-Migration: je Steuersatz, alte Positionsrundung und ungeklärt wie `Betragssnapshot`, Steuerzeilen in `dokumentsteuer`
REM-12	Nicht entfernbare Temp-Verzeichnisse (Upgradeprobe, Gesamtexport) führen nur zu einer Warnung
ERE-09	Misch- und Nullsteuersätze bestehen Mustang-Prüfung
ERE-10	Abweichende Beträge/Einheiten und externe Entitäten blockiert
ERE-11	Normfehler überschreiben kein vorhandenes Ziel
GUI-REM-01	Ungültiger Datumstext bleibt nach Commit/Fokuswechsel sichtbar
GUI-REM-02	Numerische/chronologische Sortierung, null stets zuletzt
GUI-REM-03	Einstellungen: Speichern/Verwerfen/Abbrechen und Validierungsfehler
GUI-REM-04	Rückfrage beim Beenden auch, wenn die Einstellungen gerade nicht angezeigt werden
PDF-REM-01	Große Beträge, Stapellayout, Langtexte, Folgeseiten; Zeichenkoordinaten und gerenderte Prüfbilder
PDF-REM-02	Altpositionen ohne Einheit/Produktnummer ohne Hinweistext; keine Position über einen Seitenwechsel getrennt (Tabelle und Stapel)
PERF-01	Spring/SQLite mit 5000 Kunden, 5000 Produkten und 1000 Belegen mit fünf Positionen

GUI- und Leistungstests sind getrennt aktivierbar. Der Release-Workflow verlangt sie zusätzlich zum regulären `verify`. Drei frische Starts messen bis zum Readiness-Marker nach Fensteranzeige und erstem JavaFX-Ereigniszyklus. Zeiten und Runnerdaten werden archiviert. Derselbe JAR-Build wird auf drei Plattformen geprüft. Der MSI-Test installiert, startet mit gebündelter Laufzeit und deinstalliert auf einem disponiblen Windows-Runner.

ID	Nachweis	Ausführung
ERE-12	Fremde Schemaquellen, Pfadmanipulation und Aufweichen der JAXP-Sperre werden abgelehnt	`KlassenpfadSchemaFactoryTest`

ID	Nachweis	Ausführung
ERE-PAKET-01	Gültiges XML und ungültiger Käufername mit unverändertem Ziel im gepackten JAR; Flyway findet dort V1–V6 inkl. Java-Migration	`PaketXmlPruefung`, Plattformmatrix

GUI-REM-03 prüft zusätzlich einen echten Datenbankfehler beim Speichern: Die Anwendung bleibt offen und die bearbeiteten Felder samt Ungespeichert-Anzeige bleiben erhalten.

Abweichend vom historischen Kapitel 2 erwartet PDF-TC-17 ab Version 3.1 unbekannte Summen bei unvollständigen Positionen. REM-05 prüft zusätzlich unbekannte Positionssteuer und unbekanntes Positionsbrutto; CSV darf daraus keine bekannte Null erzeugen.

Portfolio-Prüfungen (10.10.2026)

Diese ergänzenden Prüfprogramme verwenden das zuvor gebaute JAR und ausschließlich isolierte Testdaten. Sie werden getrennt von JUnit ausgeführt und verändern dessen Testfallzahl nicht. Befehle und erwarteter Vorführablauf stehen im Portfolio-Abschluss.

ID	Prüfung	Erwartetes Ergebnis
PORT-01	tools/portfolio-demo.ps1, produktive Dienste im gepackten JAR	Neuer isolierter Bestand; Belegkette, Preisänderung ohne Snapshotänderung; PDF, validiertes XML, Gesamt-ZIP und Sicherung
PORT-02	tools/windows-paket-pruefen.ps1	Portable EXE startet mit gebündelter Laufzeit ohne JAVA_HOME, isolierte DB und Bereitschaftsmarker; SHA-256 im Bericht

PORT-02 verwendet 30 Sekunden als reine Smoke-Frist und ersetzt weder die 5-Sekunden- Messung noch den installierten MSI-Test. Die Usability-Studie ist derzeit zurückgestellt; ihre Akzeptanzkriterien bleiben unbewiesen. Der Netzwerk-Monitoring-Nachweis ist offen.